

50 SQL Queries for Yelp Data Warehouse Project

Schema Overview

Dimension Tables

- dim_business
- dim_user
- dim_date

Fact Table

- fact_reviews

1. Total Number of Businesses

```
SELECT COUNT(*) AS total_businesses
FROM dim_business;
```

2. Total Number of Reviews

```
SELECT COUNT(*) AS total_reviews
FROM fact_reviews;
```

3. Average Business Rating

```
SELECT ROUND(AVG(CAST(business_stars AS FLOAT)), 2) AS avg_business_rating
FROM dim_business;
```

4. Total Active Users

```
SELECT COUNT(DISTINCT user_id) AS active_users
FROM fact_reviews;
```

5. Open vs Closed Businesses

```
SELECT
    is_open,
    COUNT(*) AS total_businesses
FROM dim_business
GROUP BY is_open;
```

6. Businesses by City

```
SELECT
    city,
    COUNT(*) AS total_businesses
FROM dim_business
GROUP BY city
ORDER BY total_businesses DESC;
```

7. Top 10 Businesses by Reviews

```
SELECT
    name,
    business_review_count
FROM dim_business
ORDER BY CAST(business_review_count AS INTEGER) DESC
LIMIT 10;
```

8. Highest Rated Businesses

```
SELECT
    name,
    business_stars
FROM dim_business
ORDER BY CAST(business_stars AS FLOAT) DESC
LIMIT 10;
```

9. Average Rating by City

```
SELECT
    city,
    ROUND(AVG(CAST(business_stars AS FLOAT)), 2) AS avg_rating
FROM dim_business
GROUP BY city
ORDER BY avg_rating DESC;
```

10. Cities with Most Reviews

```
SELECT
    b.city,
    COUNT(*) AS total_reviews
FROM fact_reviews f
JOIN dim_business b
ON f.business_id = b.business_id
GROUP BY b.city
ORDER BY total_reviews DESC;
```

11. Review Distribution by Stars

```
SELECT
    review_stars,
    COUNT(*) AS total_reviews
FROM fact_reviews
```

```
GROUP BY review_stars
ORDER BY review_stars;
```

12. Percentage Distribution of Ratings

```
SELECT
    review_stars,
    COUNT(*) AS total_reviews,
    ROUND(
        COUNT(*) * 100.0 /
        (SELECT COUNT(*) FROM fact_reviews),
        2
    ) AS percentage_distribution
FROM fact_reviews
GROUP BY review_stars;
```

13. Yearly Review Growth

```
SELECT
    d.year,
    COUNT(*) AS total_reviews
FROM fact_reviews f
JOIN dim_date d
ON f.date_key = d.date_key
GROUP BY d.year
ORDER BY d.year;
```

14. Monthly Review Trends

```
SELECT
    d.month_name,
    COUNT(*) AS total_reviews
FROM fact_reviews f
JOIN dim_date d
ON f.date_key = d.date_key
```

```
GROUP BY d.month_name
ORDER BY total_reviews DESC;
```

15. Most Useful Reviews

```
SELECT
    review_id,
    useful
FROM fact_reviews
ORDER BY CAST(useful AS INTEGER) DESC
LIMIT 10;
```

16. Average Useful, Funny and Cool Votes

```
SELECT
    ROUND(AVG(CAST(useful AS FLOAT)), 2) AS avg_useful,
    ROUND(AVG(CAST(funny AS FLOAT)), 2) AS avg_funny,
    ROUND(AVG(CAST(cool AS FLOAT)), 2) AS avg_cool
FROM fact_reviews;
```

17. Total Engagement

```
SELECT
    SUM(
        CAST(useful AS INTEGER) +
        CAST(funny AS INTEGER) +
        CAST(cool AS INTEGER)
    ) AS total_engagement
FROM fact_reviews;
```

18. Top Engaged Businesses

```
SELECT
    b.name,
    SUM(
        CAST(f.useful AS INTEGER) +
        CAST(f.funny AS INTEGER) +
        CAST(f.cool AS INTEGER)
    ) AS total_engagement
FROM fact_reviews f
JOIN dim_business b
ON f.business_id = b.business_id
GROUP BY b.name
ORDER BY total_engagement DESC
LIMIT 10;
```

19. Most Active Users

```
SELECT
    u.name,
    COUNT(*) AS total_reviews
FROM fact_reviews f
JOIN dim_user u
ON f.user_id = u.user_id
GROUP BY u.name
ORDER BY total_reviews DESC
LIMIT 10;
```

20. Elite vs Normal User Ratings

```
SELECT
    CASE
        WHEN elite IS NOT NULL AND elite != ''
        THEN 'Elite Users'
        ELSE 'Normal Users'
    END AS user_type,

    ROUND(AVG(CAST(review_stars AS FLOAT)), 2) AS avg_rating
```

```
FROM fact_reviews f
JOIN dim_user u
ON f.user_id = u.user_id
GROUP BY user_type;
```

21. Businesses with Above Average Ratings (Subquery)

```
SELECT
    name,
    business_stars
FROM dim_business
WHERE CAST(business_stars AS FLOAT) >
(
    SELECT AVG(CAST(business_stars AS FLOAT))
    FROM dim_business
);
```

22. Users with More Reviews than Average (Subquery)

```
SELECT
    name,
    user_review_count
FROM dim_user
WHERE CAST(user_review_count AS INTEGER) >
(
    SELECT AVG(CAST(user_review_count AS INTEGER))
    FROM dim_user
);
```

23. Businesses with Highest Average Review Ratings

```
SELECT
    b.name,
    ROUND(AVG(CAST(f.review_stars AS FLOAT)), 2) AS avg_review_rating
FROM fact_reviews f
JOIN dim_business b
ON f.business_id = b.business_id
GROUP BY b.name
ORDER BY avg_review_rating DESC
LIMIT 10;
```

24. Average Reviews Per User

```
SELECT
    ROUND(AVG(CAST(user_review_count AS FLOAT)), 2) AS avg_reviews_per_user
FROM dim_user;
```

25. Businesses with Low Ratings but High Reviews

```
SELECT
    name,
    business_stars,
    business_review_count
FROM dim_business
WHERE CAST(business_review_count AS INTEGER) > 500
AND CAST(business_stars AS FLOAT) < 2.5;
```

26. Top Cities by Average Ratings

```
SELECT
    city,
    ROUND(AVG(CAST(business_stars AS FLOAT)), 2) AS avg_rating,
```



```
    COUNT(*) AS total_businesses
FROM dim_business
GROUP BY city
HAVING COUNT(*) > 10
ORDER BY avg_rating DESC;
```

27. Review Count by Quarter

```
SELECT
    quarter,
    COUNT(*) AS total_reviews
FROM fact_reviews f
JOIN dim_date d
ON f.date_key = d.date_key
GROUP BY quarter;
```

28. Most Popular Categories

```
SELECT
    categories,
    COUNT(*) AS total_businesses
FROM dim_business
GROUP BY categories
ORDER BY total_businesses DESC
LIMIT 10;
```

29. Average User Fans

```
SELECT
    ROUND(AVG(CAST(fans AS FLOAT)), 2) AS avg_fans
FROM dim_user;
```

30. Top Users by Fans

```
SELECT
    name,
    fans
FROM dim_user
ORDER BY CAST(fans AS INTEGER) DESC
LIMIT 10;
```

31. Running Total of Reviews by Year (Window Function)

```
SELECT
    d.year,
    COUNT(*) AS yearly_reviews,

    SUM(COUNT(*)) OVER(
        ORDER BY d.year
    ) AS running_total_reviews

FROM fact_reviews f
JOIN dim_date d
ON f.date_key = d.date_key
GROUP BY d.year;
```

32. Rank Businesses by Ratings (Window Function)

```
SELECT
    name,
    business_stars,

    RANK() OVER(
        ORDER BY CAST(business_stars AS FLOAT) DESC
    ) AS rating_rank

FROM dim_business;
```

33. Dense Rank Users by Review Count

```
SELECT
    name,
    user_review_count,

    DENSE_RANK() OVER(
        ORDER BY CAST(user_review_count AS INTEGER) DESC
    ) AS review_rank

FROM dim_user;
```

34. Top Rated Business per City (Window Function)

```
SELECT *
FROM
(
    SELECT
        city,
        name,
        business_stars,

        ROW_NUMBER() OVER(
            PARTITION BY city
            ORDER BY CAST(business_stars AS FLOAT) DESC
        ) AS rn

    FROM dim_business
)
WHERE rn = 1;
```

35. Average Rating Difference from Overall Average

```
SELECT
    name,
    business_stars,
```

```
ROUND(  
    CAST(business_stars AS FLOAT) -  
    (  
        SELECT AVG(CAST(business_stars AS FLOAT))  
        FROM dim_business  
    ),  
    2  
) AS difference_from_avg  
  
FROM dim_business;
```

36. CTE for Business Engagement Categories

```
WITH engagement_cte AS  
(  
    SELECT  
        name,  
        business_review_count,  
        business_stars,  
  
        CASE  
            WHEN CAST(business_review_count AS INTEGER) > 500  
            AND CAST(business_stars AS FLOAT) >= 4  
            THEN 'Very High Engagement'  
  
            WHEN CAST(business_review_count AS INTEGER) < 50  
            AND CAST(business_stars AS FLOAT) >= 4  
            THEN 'Low Engagement but Good Ratings'  
  
            ELSE 'Average Engagement'  
        END AS engagement_category  
  
    FROM dim_business  
)  
  
SELECT *  
FROM engagement_cte;
```

37. Count Businesses by Engagement Category

```
WITH engagement_cte AS
(
    SELECT
        CASE
            WHEN CAST(business_review_count AS INTEGER) > 500
            AND CAST(business_stars AS FLOAT) >= 4
            THEN 'Very High Engagement'

            WHEN CAST(business_review_count AS INTEGER) < 50
            AND CAST(business_stars AS FLOAT) >= 4
            THEN 'Low Engagement but Good Ratings'

            ELSE 'Average Engagement'
        END AS engagement_category

    FROM dim_business
)

SELECT
    engagement_category,
    COUNT(*) AS total_businesses
FROM engagement_cte
GROUP BY engagement_category;
```

38. Top 5 Cities by Engagement

```
SELECT
    b.city,

    SUM(
        CAST(f.useful AS INTEGER) +
        CAST(f.funny AS INTEGER) +
        CAST(f.cool AS INTEGER)
    ) AS total_engagement

FROM fact_reviews f
JOIN dim_business b
ON f.business_id = b.business_id
GROUP BY b.city
```

```
ORDER BY total_engagement DESC
LIMIT 5;
```

39. Reviews Per Day Name

```
SELECT
    d.day_name,
    COUNT(*) AS total_reviews
FROM fact_reviews f
JOIN dim_date d
ON f.date_key = d.date_key
GROUP BY d.day_name;
```

40. Users Giving Highest Average Ratings

```
SELECT
    u.name,
    ROUND(AVG(CAST(f.review_stars AS FLOAT)), 2) AS avg_rating_given
FROM fact_reviews f
JOIN dim_user u
ON f.user_id = u.user_id
GROUP BY u.name
HAVING COUNT(*) > 50
ORDER BY avg_rating_given DESC;
```

41. Businesses with Most Cool Votes

```
SELECT
    b.name,
    SUM(CAST(f.cool AS INTEGER)) AS total_cool_votes
FROM fact_reviews f
JOIN dim_business b
ON f.business_id = b.business_id
GROUP BY b.name
ORDER BY total_cool_votes DESC
LIMIT 10;
```

42. Businesses with Most Funny Votes

```
SELECT
    b.name,
    SUM(CAST(f.funny AS INTEGER)) AS total_funny_votes
FROM fact_reviews f
JOIN dim_business b
ON f.business_id = b.business_id
GROUP BY b.name
ORDER BY total_funny_votes DESC
LIMIT 10;
```

43. Businesses with Most Useful Votes

```
SELECT
    b.name,
    SUM(CAST(f.useful AS INTEGER)) AS total_useful_votes
FROM fact_reviews f
JOIN dim_business b
ON f.business_id = b.business_id
GROUP BY b.name
ORDER BY total_useful_votes DESC
LIMIT 10;
```

44. Monthly Average Ratings

```
SELECT
    d.month_name,
    ROUND(AVG(CAST(f.review_stars AS FLOAT)), 2) AS avg_rating
FROM fact_reviews f
JOIN dim_date d
ON f.date_key = d.date_key
GROUP BY d.month_name;
```

45. Users with Elite Status

```
SELECT
    COUNT(*) AS elite_users
FROM dim_user
WHERE elite IS NOT NULL
AND elite != '';
```

46. Average Ratings by Quarter

```
SELECT
    d.quarter,
    ROUND(AVG(CAST(f.review_stars AS FLOAT)), 2) AS avg_rating
FROM fact_reviews f
JOIN dim_date d
ON f.date_key = d.date_key
GROUP BY d.quarter;
```

47. Businesses Above City Average Ratings (Correlated Subquery)

```
SELECT
    b1.name,
    b1.city,
    b1.business_stars
FROM dim_business b1
WHERE CAST(b1.business_stars AS FLOAT) >
(
    SELECT AVG(CAST(b2.business_stars AS FLOAT))
    FROM dim_business b2
    WHERE b1.city = b2.city
);
```


48. NTILE Segmentation of Users by Reviews

```
SELECT
    name,
    user_review_count,

    NTILE(4) OVER(
        ORDER BY CAST(user_review_count AS INTEGER) DESC
    ) AS user_segment

FROM dim_user;
```

49. Lag Function for Yearly Review Growth

```
SELECT
    year,
    total_reviews,

    LAG(total_reviews) OVER(
        ORDER BY year
    ) AS previous_year_reviews

FROM
(
    SELECT
        d.year,
        COUNT(*) AS total_reviews
    FROM fact_reviews f
    JOIN dim_date d
    ON f.date_key = d.date_key
    GROUP BY d.year
);
```

50. Year-over-Year Review Growth Percentage

```
WITH yearly_reviews AS
(
    SELECT
        d.year,
```

```

        COUNT(*) AS total_reviews
    FROM fact_reviews f
    JOIN dim_date d
    ON f.date_key = d.date_key
    GROUP BY d.year
)

SELECT
    year,
    total_reviews,

    LAG(total_reviews) OVER(
        ORDER BY year
    ) AS previous_year_reviews,

    ROUND(
        (
            total_reviews -
            LAG(total_reviews) OVER(ORDER BY year)
        ) * 100.0 /
        LAG(total_reviews) OVER(ORDER BY year),
        2
    ) AS yoy_growth_percentage
FROM yearly_reviews;

```